Proof-of-Change

- Proof-of-Change
  - 1. What it is
  - 2. How a check works
  - 3. Verdicts and safety caps
  - 4. Models
  - 5. Sandboxes
  - 6. Keys and privacy
  - 7. Tavily
  - 8. Ways to use it
  - 9. Security table
  - 10. What you need to run it
  - 11. Testing and CI
  - 12. Known limitations
  - 13. Tech stack and licence

Proof-of-Change

Checked against main commit `ed339426a5839443657f601e188980a89624dae8` (21 October 2026).

Each statement below is **Live** (it matches that commit, with a file and line) or **Not built**.

1. What it is

Proof-of-Change checks whether a pull request does what it claims. It reports each claim as kept (the code word is proven), broken (the code word is false claim), or could not verify, and it keeps the log line behind the statement. It can read GitHub. A person makes the final call. It never approves a pull request and it never merges one. Live: `poc/verdict.py:20` (proven and false claim), `poc/verdict.py:3` (verdicts do not block a merge), and `README.md:10` (it only gives advice).

2. How a check works

A check compares the library before the change with the library after the change. The usual path is four steps.

1. **Known-answer suite.** Eighteen small made-up cases live in `evals/bench12/cases/`. They run when the tool itself changes, through the test `tests/test_bench12.py`. They do not run before each person's check. Live: `evals/bench12/README.md:12`.

2. **Behaviour probes.** A model reads the pull-request text, writes short programs (probes), and those programs run on both sides. Live: `poc/model_provider.py:32` (claims and probes use Super) and `ops/daily_run.py:2728` (the verdict then uses the probe results).
3. **Human review.** On a console check this step is on unless the person turns it off for that check. They can keep or drop the model's probes. They can add up to 20 test ideas, each up to 500 characters. They cannot edit a probe's text. Live: `poc/console_api.py:811`, `poc/hitl.py:21`, `poc/hitl.py:22`.
4. **Tests, then the verdict.** The project's tests run before and after the change, three times on each side unless a shorter sample run is asked for. The verdict is chosen from those results and from the rules in section 3. Live: `README.md:7`, `ops/daily_run.py:2756`.

The GitHub App starts a check with the advisory swarm on and does not turn human review on. Live: `poc/github_app.py:295`. Request-a-check does the same: swarm on, and it clears the human-review variables. Live: `ops/request_worker.py:11`, `ops/request_worker.py:416`. A study run is fully automatic. Human review is refused on a study run. Live: `poc/hitl.py:3`.

3. Verdicts and safety caps

The four verdicts are `verified`, `needs_attention`, `could_not_verify`, and `no_code_change`. Live: `poc/verdict.py:11`.

These rules can only move a `verified` result downward. They do not raise a lower result to `verified`, except the setup recovery step named below, and that step has its own refusal.

- A test that passes on every base run and fails on some pull-request runs is a finding, so the result is needs attention. Live: `ops/daily_run.py:2800`, `ops/daily_run.py:2854`.
- If the base is stable and every pull-request run is a different result, the check says could not verify. It will not call that a clean pass. Live: `poc/diff.py:82`, `ops/daily_run.py:2877`.
- A test that wobbles on both sides, or only on the base, is listed and does not decide. Live: `ops/daily_run.py:2771`, `poc/diff.py:70`.
- A behaviour claim left with no probe cannot stay verified. Live: `poc/behaviour_claims.py:7`, `ops/daily_run.py:2711`.
- On a product check (`--product-check`), missing coverage caps verified at could not verify. Live: `poc/coverage_gate.py:202`, `poc/coverage_gate.py:133`.
- On a product check, any compiled file in the change (C, C++, Cython, Fortran) is named in the reason and caps verified. Live: `poc/coverage_gate.py:196`, `poc/coverage_gate.py:134`.
- After a setup retry, probe-line coverage is measured again only when the change is 200 lines or fewer. A larger change stays could not verify. A measurement that does not show a changed line ran also stays could not verify. Live: `poc/demo_helper.py:32`, `poc/demo_helper.py:1361`, `poc/demo_helper.py:1388`.
- In full mode, verified needs an AI-written probe that actually ran a changed line. Live: `ops/daily_run.py:2343`.

The advisory reviewers (the swarm) do not change the verdict at all. They cannot raise it. Live: `poc/agents.py:108`.

A docstring-only change is marked verified on this commit when it is not already needs attention, with the reason that behaviour claims hold by construction. Live: `ops/daily_run.py:2714`. Draft pull request 56 is still open and is not part of this commit.

4. Models

All four models run on NVIDIA Nemotron through Nebius Token Factory. Each call uses temperature 0 and strict JSON. Live: `poc/model_provider.py:174`.

Model	Job on this commit	Why this size
Nemotron 3 Super 120B, 12B active (`nvidia/nemotron-3-super-120b-a12b`)	Reads claims, writes probes, writes the summary. This is the only model the verdict rules consult.	The check calls it on every live probe path, and full mode refuses verified unless one of its probes ran a changed line. Live: `poc/model_provider.py:32`, `ops/daily_run.py:2343`.
Nemotron 3 Ultra 550B, 55B active (`nvidia/Nemotron-3-Ultra-550b-a55b`)	Advisory second opinion. It labels findings. It does not own the verdict.	It runs after the verdict, so a larger model can give a label without being allowed to change the result. Live: `poc/agents.py:4`, `poc/agents.py:149`.
Nemotron 3.5 Lightning (`nvidia/Nemotron-3_5-Lightning`)	Short notes on install problems and noisy tests. Advisory only.	The notes are optional and capped at five. They are not findings. Live: `poc/agents.py:47`, `poc/model_provider.py:35`.
Nemotron 3 Nano 30B, 3B active (`nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B`)	One retry if Lightning's call fails. The next check tries Lightning again.	It exists only as that fallback. Live: `poc/model_provider.py:38`, `poc/agents.py:177`.

5. Sandboxes

Pull-request code is meant to run inside a sandbox.

- **Contree** (Nebius cloud sandboxes) is preferred when the person's own Nebius key and project ID are present and the Contree library is installed. The run is billed to that account. Live: `poc/console_api.py:214`, `README.md:64`.
- **Bubblewrap** is the Linux fallback when Contree cannot run. Live: `poc/console_api.py:214`.
- **Contree-only mode.** Installs, tests, and probes run in Contree. A local step that would run pull-request code is refused. Live: `ops/daily_run.py:3401`, `tests/test_contree_only.py:87`.

- There is no automatic unsandboxed fallback. The only opt-in is the host setting `POC_UNSAFE_NO_SANDBOX=1`. Live: `poc/console_api.py:177`, `docs/decisions/2026-10-21-console-refuses-unsandboxed.md:16`.
- A product check in Contree runs pytest with coverage inside the pull-request checkpoint and reads the report back. If that file is missing, the result stays could not verify. Live: `ops/daily_run.py:2254`, `ops/daily_run.py:3458`.
- Contree-only mode does not measure which lines a probe ran. A full check still needs that measurement, so a full Contree-only check stays could not verify. Live: `ops/daily_run.py:3464`, `ops/daily_run.py:2343`.

6. Keys and privacy

- Keys typed into the console stay in the browser tab's session storage. They are not put in local storage or cookies. Live: `console/web/src/startForm.tsx:55`.
- On the server they stay in memory for that session. Live: `poc/console_api.py:701`.
- They are not passed to pull-request code. The environment that untrusted code receives is an allowlist, and credential-like names are rejected. Live: `guardrails/isolation.py:61`, `guardrails/isolation.py:86`. The Contree coverage command is built without a key. Live: `ops/daily_run.py:2258`.
- A user check never uses the server's Nebius key. The server value is dropped from the child environment and kept only so logs can be scrubbed. Live: `poc/console_api.py:57`, `poc/console_api.py:73`.
- A check with no typed key is refused before anything runs, with the words "Add your Nebius API key to run a live check." Only the saved sample may run without a key. Live: `poc/console_api.py:662`, `poc/console_api.py:731`.
- A check started from `poc/serverless.py` does not pass a session, so it has no typed key. It is refused unless it is that sample. Live: `poc/serverless.py:54`. This is recorded here and is not changed.
- Each signed-in person sees their own checks. An administrator also sees runs that have no owner. Live: `poc/console_api.py:344`.
- Sign-in is a password plus an emailed code, or optional Google or GitHub. Live: `poc/console_auth.py:1`.

7. Tavily

Tavily can suggest package names. A suggestion is installed only when that package's PyPI page links back to the same GitHub repository, and the link must be http or https. Live: `poc/demo_helper.py:6`, `poc/demo_helper.py:930`, `poc/demo_helper.py:956`. The package author writes that link. It stops an accident. It does not stop someone who is trying to get a bad package installed. Live: `docs/decisions/2026-10-21-pr-only-flaky-not-verified.md:11`, `README.md:108`.

The retry rule, in the program's own words: "If setup fails, the helper retries the install up to 2 times. If a test still can't be verified because something is missing, it installs what's missing and reruns that test once." Live: `poc/demo_helper.py:24`.

Related bug reports are background. The report labels them as not used for the verdict. They are not fetched when the swarm will not run. Live: `poc/related_reports.py:8`, `poc/related_reports.py:22`.

8. Ways to use it

- **Command line.** `ops/daily_run.py` with a `--pr` link. Live: `poc/console_api.py:166` (the console builds that same command).
- **Web console.** React pages in `console/web`, API in `poc/console_api.py`.
- **Request a check.** A public GitHub issue. The default cap is five checks a day (`POC_DAILY_CAP`). Live: `ops/request_worker.py:63`, `README.md:58`.
- **GitHub App.** Swarm is always on for an App check. Live: `poc/github_app.py:298`. It posts a check and a comment only on allowlisted demo repositories when a token is set. Live: `README.md:11`.
- **Results page.** https://aritracodesbad123.github.io/Proof-of-change/
- **Codespaces.** Live checks in a Codespace have not been confirmed to work yet. Live: `README.md:62`. The steps under that heading are for trying it anyway. A live check there needs the person's own Nebius key and project ID.

9. Security table

Protection	Status	Where
Verdicts do not merge or block	Live	`poc/verdict.py:3`, `poc/verdict.py:37`
Coverage gate and AI-probe gate only lower verified	Live	`poc/coverage_gate.py:5`, `ops/daily_run.py:2343`
Setup recovery can raise could not verify only after a small-change remeasure	Live	`poc/demo_helper.py:1388`
Swarm does not change the verdict	Live	`poc/agents.py:108`
No key in the untrusted environment	Live	`guardrails/isolation.py:61`
User checks ignore the server key	Live	`poc/console_api.py:57`, `poc/console_api.py:79`
No key, no live check	Live	`poc/console_api.py:731`

Protection	Status	Where
Serverless start has no session	Live	`poc/serverless.py:54`
Keys scrubbed from helper text	Live	`poc/demo_helper.py:35` (secret names), `poc/related_reports.py:69`
Bubblewrap sandbox	Live	`guardrails/sandbox.py`, used from `ops/daily_run.py`
Contree billed to the typed account	Live	`README.md:64`
No unsandboxed fallback except `POC_UNSAFE_NO_SANDBOX=1`	Live	`poc/console_api.py:177`
PyPI link-back before install	Live	`poc/demo_helper.py:956`
Related reports are not a verdict input	Live	`poc/related_reports.py:22`
Each person sees their own checks	Live	`poc/console_api.py:344`
A process that double-forks can escape the timeout kill	Live limitation	`docs/decisions/2026-10-21-install-timeout-kills-process-tree.md:20`
Link-back is not attack-proof	Live limitation	`docs/decisions/2026-10-21-pr-only-flaky-not-verified.md:35`

10. What you need to run it

The core package lists no required third-party libraries. It shells out to `uv` and `git`. Live: `pyproject.toml:6`.

Figures supplied for this write-up, measured on Linux. This commit does not contain the measurement log.

- A sample check took 42 seconds, with 398 MB peak memory.
- A live check took 4 minutes 6 seconds, with 708 MB peak memory and 4.3 cores at peak.
- Peak working disk was 742 MB. The package cache is about 1 GB.

Practical minimum from those figures: 2 GB of RAM, 2 cores (4 cores if you want full speed), and 3 GB of free disk.

11. Testing and CI

On every pull request, GitHub Actions runs, on Python 3.11 and 3.12: a decision-log check, lint (`ruff`), a safety scan that forbids external side effects, the test suite (which includes the eighteen

bench12 cases), and the golden evals. A separate job builds the console. Live: `.github/workflows/ci.yml:56` through `.github/workflows/ci.yml:91`, and `evals/bench12/README.md:12`.

12. Known limitations

- The runner installs and tests Python projects (`pyproject.toml`, pytest). It is not a checker for other languages. Live: `pyproject.toml:6`, `README.md:7`.
- A full Contree-only check cannot reach verified, because probe coverage is not measured there. A product check can reach verified when the coverage report comes back and the other caps allow it. Live: `ops/daily_run.py:3464`, `docs/decisions/2026-10-21-contree-coverage.md:5`.
- The PyPI link-back check is not attack-proof. The package author writes the link, so someone trying to get past it can write one too. Live: `docs/decisions/2026-10-21-pr-only-flaky-not-verified.md:35`.
- Docstring-only changes are marked verified on this commit when they are not already needs attention. Draft pull request 56 is open and is not merged here. Live: `ops/daily_run.py:2714`.
- A process that double-forks and is adopted by init can escape the install timeout. Live: `docs/decisions/2026-10-21-install-timeout-kills-process-tree.md:20`.
- Codespace live checks are not confirmed. Live: `README.md:62`.
- A managed schedule for the serverless daily job is not built. Live: `docs/REQUIREMENTS.md:286` (“Planned, not yet built” for that schedule).

13. Tech stack and licence

- Python core, no required outside libraries (`pyproject.toml:6`). Optional extras add pytest, ruff, the Contree SDK, and PyJWT for the GitHub App (`pyproject.toml:8`).
- `uv` creates the environments that run a pull request (`pyproject.toml:10`).
- The console is React, TypeScript, and Vite (`console/web`).
- Sandboxes: Contree and bubblewrap (section 5).
- Docker and Render host the console (`Dockerfile`, `render.yaml`).
- GitHub Actions runs CI (section 11).
- The licence is MIT. Live: `LICENSE:1`.